

Execution Guide

All scripts are run from the **project root** (language-quality-and-publication-success/). All input and output paths are relative to that directory.

Prerequisites

Python version

Python **3.10 or later** is required.

Installation

Create and activate a virtual environment, then install dependencies:

```
python -m venv .venv
# Linux/macOS:
source .venv/bin/activate
# Windows:
.venv\Scripts\activate

pip install pandas numpy scikit-learn scipy matplotlib
pip install py-readability-metrics
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
# CPU-only
# OR for GPU (replace cu121 with your CUDA version):
# pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu121
pip install transformers
pip install spacy
pip install lightgbm xgboost statsmodels
pip install pymupdf pymupdf4llm # only needed for pdf2txt_with_fitz.py
pip install vllm # only needed for
qwen_native_like_prediction.py; GPU cluster only
```

Runtime requirements by stage

Stage	CPU/GPU	RAM	Approx. time
1 — Sampling	CPU	4 GB	~5 min
2 — Abstract cleaning	CPU	4 GB	~5 min
3–4 — PDF/JSON conversion	CPU	2 GB	varies
5 — Abstract readability	CPU	4 GB	~2 h (all 50 shards)
5 — Abstract perplexity	GPU recommended	8 GB RAM + 8 GB VRAM	~1 h/shard
6 — Paper readability	CPU	4 GB	varies
6 — Grammar detection	GPU recommended	16 GB RAM + 8 GB VRAM	varies

Stage	CPU/GPU	RAM	Approx. time
6 — Native-likeness (Qwen 14B)	GPU required	32 GB RAM + 24 GB VRAM	varies
7 — Validation	CPU	2 GB	< 1 min
8–9 — Preprocessing / splits	CPU	8 GB	~5 min
10 — EDA	CPU	8 GB	~10 min
11 — Model selection	CPU	8 GB	~30–60 min
12 — Model training (per script)	CPU	8 GB	~5–20 min
13 — Ablation (25 models each)	CPU	8 GB	~2–4 h each
14 — Feature importance	CPU	2 GB	< 1 min

Stage 1 — Sampling (optional; 90k sample already in data/)

Draws a stratified 90k sample from the full arXiv metadata.

```
python code/01_data_collection/stratified_sampler.py
```

- **Input:** data/full_arxiv_before_sampling.csv
 - **Output:** data/arxiv_metadata_stratified_90k.csv
-

Stage 2 — Abstract cleaning (optional; cleaned data already in data/)

Cleans and deduplicates abstracts.

```
python code/01_data_collection/clean_abstracts.py
```

- **Input:** data/90k_arxiv_metadata_from_semantic_scholar.csv
 - **Output:**
data/90k_arxiv_metadata_from_semantic_scholar_cleaned.csv
 - **Audit log:**
data/90k_arxiv_metadata_from_semantic_scholar_removed.csv
-

Stage 3 — PDF → JSON conversion (requires paper PDFs)

Converts a single PDF to JSON. Paper PDFs are **not included** in this submission.

```
python code/01_data_collection/pdf2txt_with_fitz.py \  
  --input PATH/T0/paper.pdf \  
  --output PATH/T0/output_dir/
```

Stage 4 — JSON → TXT conversion (requires paper JSONs from Stage 3)

Converts paper JSONs to plain-text files. Run one of the following modes:

```
# Single file  
python code/01_data_collection/json2txt.py \  
  --in_json PATH/T0/paper.json \  
  --out_dir PATH/T0/txt_output/  
  
# Whole directory  
python code/01_data_collection/json2txt.py \  
  --in_dir PATH/T0/json_dir/ \  
  --out_dir PATH/T0/txt_output/  
  
# From a list file (recommended for large batches)  
python code/01_data_collection/json2txt.py \  
  --list_file PATH/T0/file_list.txt \  
  --out_dir PATH/T0/txt_output/
```

Stage 5 — Feature extraction on abstracts (optional; features already in full CSVs)

Readability metrics (abstracts)

Runs as 50 shards via SLURM_ARRAY_TASK_ID; defaults to shard 0 locally.

```
python code/02_feature_extraction/readability_on_arxiv_abstracts.py
```

- **Input:** data/90k_arxiv_metadata_from_semantic_scholar.csv
- **Output:**
data/abstract_readability_50_shards/abstract_readability_shard_00.csv (and one file per shard)

Perplexity (abstracts)

Runs as 50 shards; defaults to shard 0 locally. Requires a GPU for reasonable speed.

```
python code/02_feature_extraction/perplexity_arxiv_abstracts.py \  
  --array_index 0 \  
  --num_shards 1
```

- **Input:** data/90k_arxiv_metadata_from_semantic_scholar.csv
 - **Output:** data/abstract_gpt2_ppl_outputs_50shards/
-

Stage 6 — Feature extraction on full papers (requires paper TXTs from Stage 4)

Paper TXT files are **not included** in this submission. Pass the directory containing them via CLI arguments.

Readability metrics (papers)

```
python code/02_feature_extraction/readability_on_arxiv_papers.py \
  --input_dir PATH/T0/txt_dir/ \
  --output_csv results/paper_readability.csv \
  --split train # or test
```

Grammar error detection

```
python code/02_feature_extraction/detect_grammar.py \
  --input_root PATH/T0/txt_root/ \
  --output_root results/grammar_outputs/ \
  --shard_id 0 \
  --n_shards 1
```

Or for a single file:

```
python code/02_feature_extraction/detect_grammar.py \
  --input PATH/T0/paper.txt \
  --output results/grammar_outputs/paper.json
```

Native-likeness prediction (GPU required)

```
python code/02_feature_extraction/qwen_native_like_prediction.py \
  --in_dir PATH/T0/txt_dir/ \
  --out_dir results/qwen_outputs/ \
  --model_path Qwen/Qwen2.5-14B-Instruct-AWQ
```

The 09_cluster_jobs/ sbatch scripts show how these were launched on a SLURM cluster.

Stage 7 — Validation of language quality features

Runs Mann-Whitney U tests and rank-biserial correlations on the 200-paper benchmark.

```
python
code/03_validation_of_language_quality_features/validation_of_language_quality_features.py
```

- **Input:** data/200-paper-benchmark_validation_of_linguistic_features.csv
 - **Output:** data/200-paper-benchmark/linguistic_feature_boxplots/ (plots + CSV)
-

Stage 8 — Preprocessing (optional; preprocessed data already in data/)

Merges and encodes all features into classifier-ready datasets.

```
python code/04_preprocessing/preprocess.py
```

- **Input:** data/90k_arxiv_citation_prediction_full.csv
 - **Output:** data/preprocessed_90k_classifiers/
-

Stage 9 — Generate train/val/test splits (optional; splits already in data/)

```
python code/04_preprocessing/generate_splits_citations.py  
python code/04_preprocessing/generate_splits_doi.py
```

- **Inputs:** data/90k_arxiv_citation_prediction_full.csv, data/90k_arxiv_doi_prediction_full.csv
 - **Outputs:** data/90k_arxiv_citation_prediction_splits/, data/90k_arxiv_doi_prediction_splits/
-

Stage 10 — Exploratory data analysis

```
python  
code/05_exploratory_and_correlation_analysis/eda_and_correlation_analysis.py
```

- **Inputs:** data/90k_arxiv_citation_prediction_full.csv, data/90k_arxiv_doi_prediction_full.csv
 - **Output:** results/eda_publication_success_outputs/
-

Stage 11 — Model and threshold selection

```
python code/06_model_selection/model_and_threshold_selection_for_citations.py  
python code/06_model_selection/model_selection_for_doi.py
```

- **Inputs:** split CSVs from data/90k_arxiv_citation_prediction_splits/ and data/90k_arxiv_doi_prediction_splits/
 - **Output:** results/model_selection/
-

Stage 12 — Model training and evaluation

Each script trains the model and prints evaluation metrics to stdout.

Citation count prediction (citation_count > 5)

```
python code/07_models/citation_prediction/lightgbm_citation_prediction.py
python code/07_models/citation_prediction/xgboost_citation_prediction.py
python
code/07_models/citation_prediction/logistic_regression_citation_prediction.py
```

DOI presence prediction

```
python code/07_models/doi_prediction/lightgbm_doi_prediction.py
python code/07_models/doi_prediction/xgboost_doi_prediction.py
python code/07_models/doi_prediction/logistic_regression_doi_prediction.py
```

- **Inputs:** split CSVs from data/90k_arxiv_citation_prediction_splits/ and data/90k_arxiv_doi_prediction_splits/
-

Stage 13 — Ablation study

```
python code/08_analysis/ablation/ablations_for_citation_prediction.py
python code/08_analysis/ablation/ablations_for_doi_prediction.py
```

- **Output:** ablation_citation_gt_5_results_xgboost.csv, ablation_doi_prediction_results_xgboost.csv (written to the working directory)
-

Stage 14 — Feature importance analysis

```
python
code/08_analysis/feature_importance/feature_importance_analysis_citations.py
python code/08_analysis/feature_importance/feature_importance_analysis_doi.py
```

- **Output:** results/feature_importance_plots/
-

Notes

- **Stages 1–2, 8–9** are optional because the processed data files are already included in data/.
- **Stages 3–4, 6** require paper PDFs or TXTs which are not included in this submission. All scripts in those stages accept input paths as command-line arguments.
- **Stage 6 (Qwen)** additionally requires a GPU and either a local copy of Qwen2.5-14B-Instruct-AWQ or a HuggingFace download.
- The 09_cluster_jobs/ folder contains SLURM batch scripts used to run grammar detection and native-like language classification on an HPC cluster. They are provided for reference only and are not needed for local execution.